Widgets et mise en page

Séance 2 · état local et formulaires

Point de situation

Séance 1 : Flutter installé, Dart parcouru, le tutoriel officiel commencé.

Aujourd'hui commence **Pantry**, l'application construite jusqu'à la fin du semestre.

Cette séance se limite à la mise en page et à la saisie : la navigation arrive en séance 3, le réseau en séance 4, l'état partagé en séance 5.

Aujourd'hui

Horaire	Bloc
0:00 – 0:15	Rappels
0:15 – 1:00	Widgets, cycle de vie, modèle de layout
1:00 – 2:00	TP 1 · Devine le rendu
2:00 – 2:15	Pause
2:15 – 2:55	`setState` et formulaires
2:55 – 3:55	TP 2 · fiche produit et formulaire
3:55 – 4:15	Débrief

Widgets

Cycle de vie et modèle de contraintes

Le `State` survit, le widget non

- **`StatelessWidget`** : ne dépend que de ses paramètres
- **`StatefulWidget`** : associé à un `State`, qui porte les données mutables
- À chaque build, le widget est recréé et le `State` conservé

Les quatre moments d'un `State`

Méthode	Quand
`initState`	une fois, à l'insertion
`didUpdateWidget`	à chaque nouveau widget parent
`build`	à chaque reconstruction
`dispose`	au retrait définitif

- `initState` est synchrone : un `Future` lancé depuis lui continue même après `dispose`
- `mounted` : vrai tant que le `State` est dans l'arbre, à tester après chaque `await`

```
final product = await repository.load(barcode);
if (!mounted) return;
setState(() => _product = product);
```

Widget, Element, RenderObject

Arbre	Rôle	Durée de vie
Widget	configuration **immuable**	recréé à chaque build
Element	instance montée, fait le lien	persistant tant que le type ne change pas
RenderObject	mesure, positionne, peint	persistant, coûteux

Un widget n'est **pas** l'objet à l'écran : c'est une description, jetable.

Un build ne repeint que ce qui a changé

Créer des widgets coûte peu. La mise en page et la peinture coûtent cher : Flutter ne les refait que là où la description a changé.

Un widget `const` n'est pas reconstruit par son parent

```
class _CounterState extends State<Counter> {
  int _count = 0;

  @override
  Widget build(BuildContext context) => Column(
        children: [
          Text('$_count'), // new instance at each setState
          const Logo(),    // same instance: skipped
        ],
      );
}
```

- `_CounterState` construit `Column`, `Text` et `Logo` : chaque `setState` relance ce `build`
- `const Logo()` est la même instance à chaque build : Flutter saute tout son sous-arbre
- Reconstruit quand même par son propre `setState`, ou par un `Theme.of` (séance 3)
- `very_good_analysis` signale chaque `const` manquant

Constraints go down.

Sizes go up.

Parent sets position.

Les contraintes descendent. Les tailles remontent. Le parent positionne.

Un enfant ne connaît jamais sa position

1. Un parent transmet à son enfant des **contraintes** : largeur et hauteur, minimales et maximales.
2. L'enfant choisit sa **taille** dans ces bornes, et la renvoie.
3. Le parent **place** l'enfant dans son propre repère.

Un parent n'impose jamais directement une taille : il impose des bornes.

Référence : `docs.flutter.dev/ui/layout/constraints`, 29 exemples commentés.

Contraintes tight, loose et unbounded

```
SizedBox(width: 100, child: …)   // tight: min = max = 100
Center(child: …)                 // loose: min = 0, max = available
```

Contrainte	`min / max`	L'enfant…
tight	`min == max`	n'a aucun choix, sa demande est ignorée
loose	`min == 0`	choisit entre 0 et le maximum
unbounded	`max == infinity`	doit se dimensionner lui-même, sinon erreur

Exemple : une `ListView` dans une `Column` lève « Vertical viewport was given unbounded height ».

`Row` et `Column` laissent l'axe principal libre

```
Row(children: [
  Expanded(child: A()),              // gets a tight constraint
  Expanded(flex: 2, child: B()),     // twice as wide
  const Icon(Icons.star),            // sizes itself
])
```

- Axe principal : contrainte **unbounded**, chaque enfant prend ce qu'il veut
- `Expanded` : partage l'espace **restant** au prorata des `flex`
- `Flexible` : même partage, mais l'enfant peut prendre moins

`Expanded` ne dessine rien

```
Row(children: [
  Padding(
    padding: const EdgeInsets.all(8),
    child: Expanded(child: A()), // parent is Padding, not Row
  ),
])
```

- Annotation posée sur un enfant, lue par son parent
- Seuls `Row` et `Column` la lisent ; dans un `Stack`, l'équivalent est `Positioned`
- Sous un autre parent : `Incorrect use of ParentDataWidget.`, qui désigne le **parent immédiat**

Quatre symptômes d'une erreur de mise en page

À l'écran	Nature
rayures jaunes et noires	dépassement de place
une zone non peinte	erreur de layout, contrainte impossible
un rendu d'apparence normale	annotation sous le mauvais parent
écran rouge	exception levée pendant un `build`

Dans les quatre cas, l'application continue de tourner, et la console nomme le widget fautif.

Lire les contraintes avant d'ajouter un widget

```dart
import 'package:flutter/rendering.dart';

void main() {
  debugPaintSizeEnabled = true;   // shows sizes and padding
  runApp(const PantryApp());
}
```

Le **Widget Inspector** de l'extension Flutter affiche l'arbre et, pour chaque nœud, les contraintes reçues et la taille renvoyée.

Utiliser l'un des deux dès le premier symptôme, avant d'ajouter un widget au hasard.

Pas de `Modifier` : chaque effet est un widget

```
Modifier.padding(16.dp).background(Color.Red)      // Compose
```

```
Padding(                                            // Flutter
  padding: const EdgeInsets.all(16),
  child: ColoredBox(color: Colors.red, child: …),
)
```

Chaque effet enveloppe le widget suivant : l'arbre est plus profond, mais tout reste un widget.

Autres correspondances : `docs.flutter.dev/flutter-for/compose-devs`.

Noms des composants Material 3 en Compose et en Flutter

Même nom en Compose et en Flutter : `Scaffold`, `Card`, `FloatingActionButton`, `NavigationBar`, `SegmentedButton`, `FilterChip`, `Switch`, `Checkbox`.

Compose	Flutter
`TopAppBar`	`AppBar`
`ListItem`	`ListTile`
`Button`	`FilledButton`
`TabRow`	`TabBar` et `TabBarView`
`ExposedDropdownMenuBox`	`DropdownMenu`
`OutlinedTextField`	`TextField` avec `OutlineInputBorder`

Chercher le composant dans `docs.flutter.dev/ui/widgets/material` avant de l'assembler à partir de `Container`.

TP 1 : Devine le rendu

60 minutes

Huit extraits. **Prédiction écrite avant exécution.**

`docs/seance-02-devine-le-rendu.md`

Pause

15 minutes

`setState`

État local et formulaires

`setState` reconstruit tout le `build()`

```
setState(() => _tabIndex = index);
```

- Exécute la fonction passée, où l'état change, puis marque le `State` à reconstruire
- Reconstruit **tout le `build()`** du `State`, pas seulement la partie modifiée ; les instances `const` sont sautées
- Placer le `State` au plus près de ce qu'il modifie

Trois limites de `setState`

- Reconstruction à la frame suivante, pas immédiatement
- Un autre `State` n'en sait rien
- Interdit après `dispose` : tester `mounted` après un `await`

Partage d'état entre écrans : séance 5.

Les trois pièces d'un formulaire

```dart
final _formKey = GlobalKey<FormState>();

Form(
  key: _formKey,
  child: TextFormField(
    validator: (v) => (v?.isEmpty ?? true) ? 'Required field' : null,
  ),
)

if (_formKey.currentState!.validate()) { … }
```

Pièce	Rôle
`Form`	regroupe les champs
`GlobalKey<FormState>`	donne accès à `validate()` et `reset()`
`validator`	renvoie `null` si valide, un message sinon

`validate()` ne vérifie que les `FormField`

- `validate()` n'interroge que les `FormField` descendants
- `showDatePicker` renvoie un `Future<DateTime?>`, `null` si l'utilisateur annule

Deux issues pour une date :

- contrôler la valeur à la main avant de soumettre ;
- envelopper le sélecteur dans un `FormField<DateTime>`.

Toujours libérer le contrôleur dans `dispose`

```dart
final _controller = TextEditingController();
final _focusNode = FocusNode();   // requestFocus() puts the cursor here

@override
void dispose() {
  _controller.dispose();
  _focusNode.dispose();
  super.dispose();
}
```

- `dispose` retire les listeners et interdit tout usage ultérieur
- Sans lui, un `State` démonté peut rester retenu en mémoire
- Même règle pour tout `ChangeNotifier`, `FocusNode` compris
- L'oubli ne produit **aucune erreur ni aucun avertissement**

Un `TextFormField` sans contrôleur fonctionne aussi : `validator` et `onSaved` suffisent souvent.

Le clavier réduit la hauteur disponible

Le `Scaffold` retire la hauteur du clavier à son `body` (`resizeToAvoidBottomInset`, `true` par défaut) : un contenu de hauteur fixe est masqué.

Approche	Effet
`ListView` pour les champs	le contenu défile, le champ actif reste atteignable
`SingleChildScrollView`	équivalent pour un contenu non listé
`resizeToAvoidBottomInset: false`	le `body` garde sa hauteur, le clavier le recouvre

Le choix du conteneur règle le problème.

Lire couleurs et styles dans `Theme.of(context)`

```dart
final scheme = Theme.of(context).colorScheme;
final text = Theme.of(context).textTheme;

Text(
  product.name,
  style: text.labelMedium?.copyWith(color: scheme.secondary),
  maxLines: 2,
  overflow: TextOverflow.ellipsis,
)
```

- Color roles et styles : mêmes noms qu'en Material 3
- Un style de `textTheme` est un `TextStyle?` : `copyWith` en change une propriété
- Une couleur en dur ignore le thème et le mode sombre (séance 3)

Rendre un TP sur son fork

Forker une fois `github.com/mduruisseau/pantry-2026` (bouton **Fork**), puis :

```
git clone https://github.com/mduruisseau/pantry-2026.git
cd pantry-2026
git remote add perso https://github.com/<compte>/pantry-2026.git

git switch session-02-start
git switch -c tp02-<nom>
…
git add -A && git commit -m "TP 2"
git push perso tp02-<nom>
```

- Séances suivantes : `git fetch origin`
- Push en HTTPS : jeton d'accès personnel ou clé SSH, pas le mot de passe du compte

Contrôle continu

- **20 %** de la note : quatre TP tirés au sort parmi les séances 2 à 10
- Rendu : la branche `tpNN-<nom>` poussée sur son fork avant la fin de la séance
- Règles complètes dans le README du dépôt du cours

TP 2 : fiche produit et formulaire

60 minutes en binôme

```
git switch session-02-start
git switch -c tp02-<nom>
flutter pub get
flutter run -d chrome
```

Énoncé complet dans `TP.md`, maquette dans `design/fiche-produit.svg`.

La fiche produit d'abord. Le formulaire peut être terminé hors séance.

Points de relecture

- les **six** produits mockés : le rendu tient-il au-delà du premier ?
- un champ absent : ligne supprimée, texte de repli, ou valeur par défaut du modèle ?
- le sélecteur de date : contrôle manuel, ou `FormField<DateTime>` ?
- les contrôleurs : qui les libère dans `dispose` ?

Erreurs fréquentes

Erreur	Effet
`ListView` dans une `Column`, sans `Expanded`	liste non peinte, « Vertical viewport was given unbounded height »
`Text` long dans une `Row`, sans `Expanded`	rayures, et `maxLines` ne tronque rien
`setState` appelé pendant un `build`	« setState() or markNeedsBuild() called during build. »
`Colors.grey` au lieu d'un color role	une couleur qui ne suit ni le thème ni le mode sombre
`main()` ou un champ du `State` modifié, puis hot reload	rien ne change : hot restart (`R`)

Séance 3

Listes, navigation, thème et transitions.

Comment un compteur dans l'AppBar connaît-il le produit affiché sur un autre écran ? `setState` n'y répond pas.

Questions

Pointeurs du cours 1

Ressource	Adresse
Contraintes	`docs.flutter.dev/ui/layout/constraints` · 29 exemples
Flutter pour développeurs Compose	`docs.flutter.dev/flutter-for/compose-devs`
Catalogue de widgets	`docs.flutter.dev/ui/widgets`
Composants Material 3	`docs.flutter.dev/ui/widgets/material`
Widget Inspector	`docs.flutter.dev/tools/devtools/inspector`

Pointeurs du cours 2

Ressource	Adresse
`Form` et validation	`api.flutter.dev/flutter/widgets/Form-class.html`
Cookbook · formulaires	`docs.flutter.dev/cookbook/forms`
`showDatePicker`	`api.flutter.dev/flutter/material/showDatePicker.html`
`Theme`	`api.flutter.dev/flutter/material/Theme-class.html`
Dépôt du cours	`github.com/mduruisseau/pantry-2026` → `session-02-start`